

1 Introduction to Parsing

In this assignment, we will be building a parser that is capable of identifying whether a computer program is valid and syntactically correct. How do we differentiate between the following syntactically correct expression

$$7 + (b * 3)$$

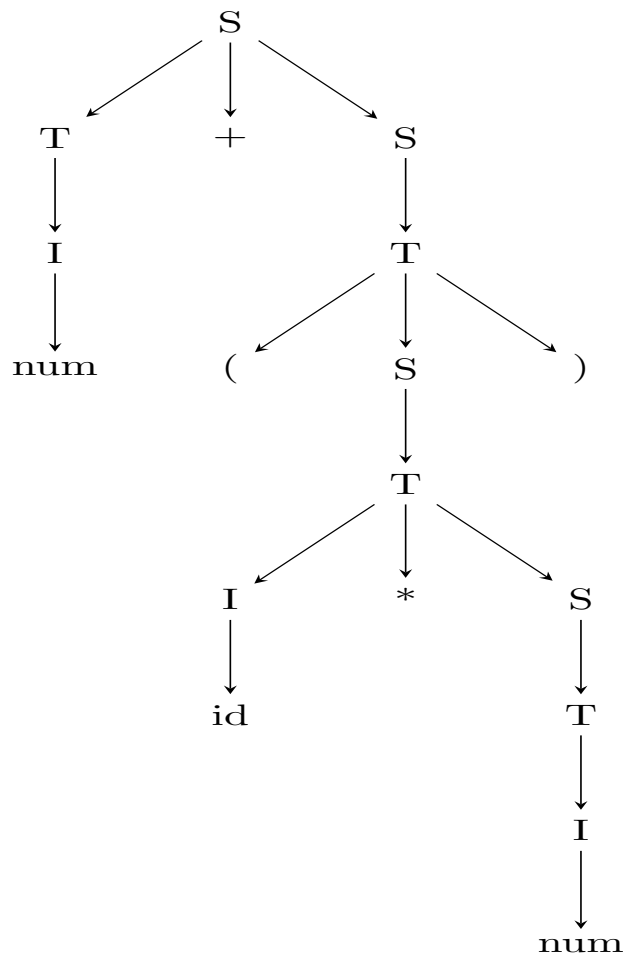
and the incorrect expression

$$7 * b * 3?$$

All expressions follow a context free grammar, which is a set of rules that can be used to derive strings. The set of all strings a grammar produces is called the language of the grammar. Consider the following grammar that is capable of producing all arithmetic expressions using additions, multiplications and brackets to enforce order of operations.

$$\begin{aligned} S &\rightarrow T \mid T + S \\ T &\rightarrow I \mid I * S \mid (S) \\ I &\rightarrow \text{id} \mid \text{num} \end{aligned}$$

The symbols S , T and I are non-terminals, while $+$, $*$, $($ and $)$ are terminal symbols. The symbols `id` and `num` are placeholders to indicate identifiers and numeric constants respectively, which can also be expressed as production rules in the grammar. But we will consider these as terminal symbols. A production rule replaces a non-terminal symbol with symbols that it derives. So the rule $S \rightarrow T + S$ replaces S with the string $T + S$. The first non-terminal symbol is the start symbol. A derivation is a set of production rules applied from the start symbol where each non-terminal is replaced with its production. A valid arithmetic expression is the set of all strings of terminal symbols that can be derived from the start symbol S . For example, the expression `num + (id * num)` has the following derivation which can be drawn as a tree. All strings that can be derived this way are syntactically valid arithmetic expressions.



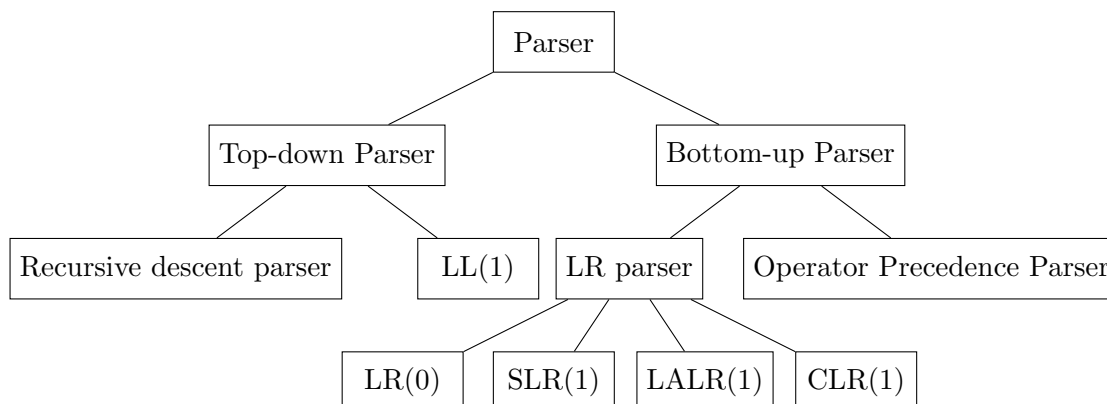
Grammars are everywhere, especially in programming languages like C++, Java and Python. A computer program written in any of these languages is a string that belongs to the respective grammar. Only, these grammars are very complicated with many non-terminals, terminal symbols and production rules. Please see <https://www.nongnu.org/hcb/> for a grammar that corresponds to the C++ programming language.

Note that although we can check for the syntax of programming languages using context-free grammars, programming languages are context-sensitive grammars (not context-free) that requires a symbol table (table of all identifiers and their types) to check for semantic errors. In this assignment however, we will only check for syntax errors.

2 Recursive Top-Down Parsers

Given an input string, how do we check whether this string belongs to a grammar, or in other words, is in the language of a grammar? There are many parsers. Some of these are Top-down parsers, where we start from the start symbol and derive the input string. Others are Bottom-up parsers, where we start from the input string and derive the start symbol. See below for a small classification of parsers. If you are interested in compiler design, parsing, program translation, compiler optimization and language design, please pick up this excellent book (sometimes called

the Dragon book): *Compilers: Principles, Techniques, & Tools*, by Aho, Lam, Sethi and Ullman.



For this assignment, we will write a simple recursive top-down parser (sometimes known as recursive decent parser). Given a grammar, we would like to accept a string as input and check whether it belongs to the grammar. So starting with the start symbol, we try one of the productions. We go down the tree as long as we replace non-terminal symbols with their productions, or if we match the input symbol with terminal symbols in the production string. If at any time the terminal symbol in the production string does not match the input string, or if there are no more rules to apply, we backtrack out of recursion and try other possibilities. We formalize this below.

Let the input string be $x = x_0x_1 \dots x_n$, and the current production string be $y = y_0y_1 \dots y_k$. Initially, $y = S$, the start symbol. Let us also suppose that in some iteration, we have matched the first $i - 1$ symbols of the input string with the first $i - 1$ symbols of the production string. Now we are comparing the symbols x_i of the input string with y_i of the production string. We have the following scenarios.

1. If $x_i = y_i$, then we have successfully matched the first i symbols of the input string. We therefore try matching the $i + 1$ symbols of both the input string and the production string. This can be checked recursively.
2. If there are no more symbols in the production string, so y_i is empty. Then we made an error earlier in the derivation. So we backtrack out of recursion.
3. If y_i is a terminal symbol, but $x_i \neq y_i$, then we backtrack out of recursion.
4. If y_i is a non-terminal symbol, we try to apply any of its production rules. We typically explore the productions in order. If the production is $y_i \rightarrow y'_0 \dots y'_j$, then the production string has y_i replaced with its production. So it becomes $y_0 \dots y_{i-1}y'_0 \dots y'_jy_{i+1} \dots y_k$. We then try matching x_i with y'_0 and the rest of the symbols in the input by recursion.

If y_i is a non-terminal symbol, and there are no more rules to apply, then we backtrack out of recursion.

Note that both the input string and the production string are scanned from left to right. See below for the first few steps of the recursion tree expansion in deriving the example string `num + (id * num)` using the grammar shown in Section 1.

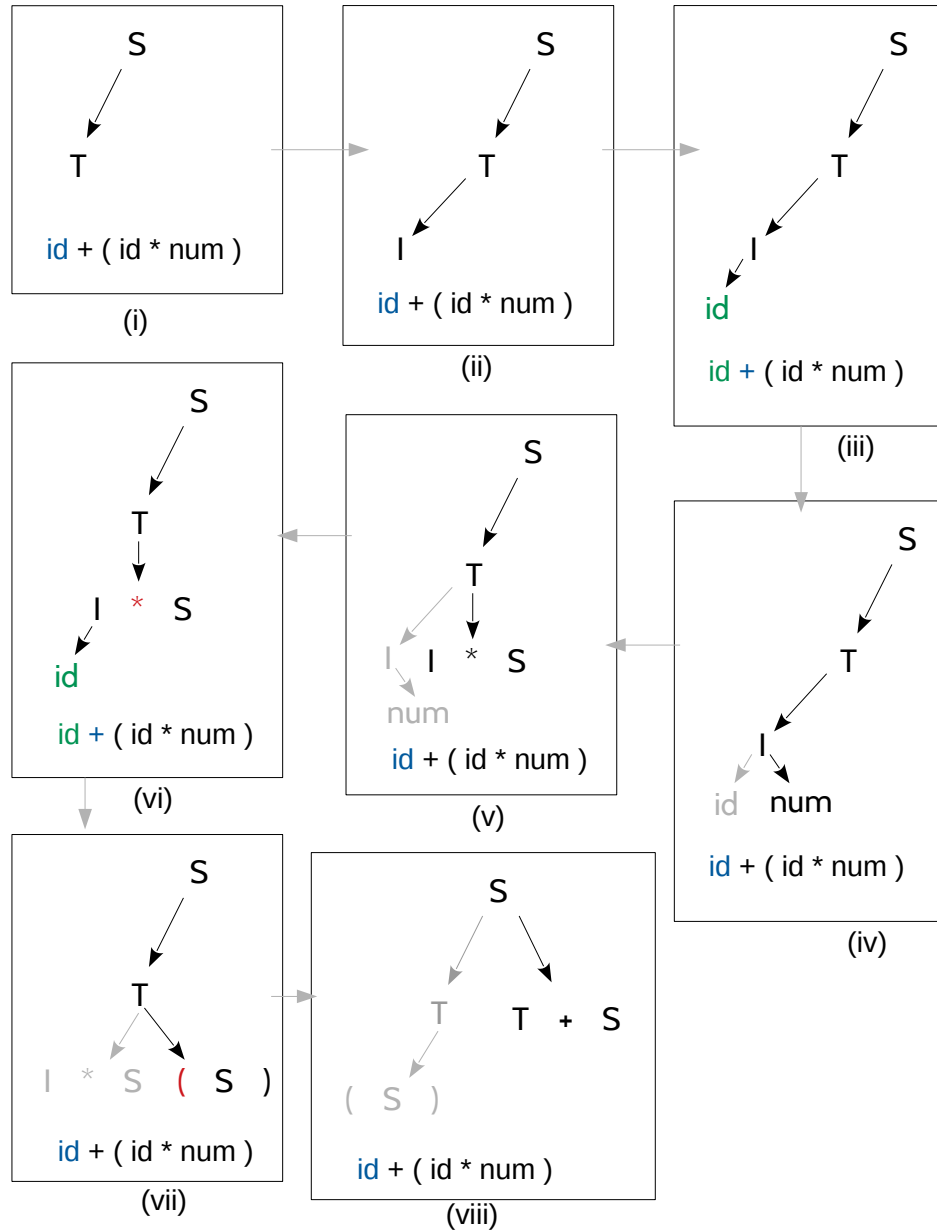


Figure: First seven steps of the recursion tree in deriving the input string `id + (id * num)`. Symbols in blue indicates current match, green indicates matched, and red indicates mismatch. Greyed regions indicate backtracking.

As can be seen, we try matching symbols of the input with the productions. In step (iii), we backtrack as there are no more symbols in the production string. In steps (vi) and (vii), we backtrack due to mismatches.

As a note, in most compilers, the input string is first broken into tokens, which are the smallest

unit of code that has meaning. The program that produces a string of tokens is called a tokenizer. The tokenizer also identifies identifiers and numeric constants. In this assignment, we will refer to all identifiers as `id` and numbers as `num`. This eliminates the need to write a more robust tokenizer, instead we split both the production string and the input string into tokens simply based on whitespaces.

3 Human Resource Machine

Steam has a wonderful programming puzzle game called Human Resource Machine. In each level of this game, we are given a simple task to transform the input stream into the desired output stream. For example, we may be given a stream of $2n$ numbers. For every two numbers x and y in the input, we may be tasked to output the remainder when x is divided by y .

In this game however, we may only perform a small set of operations as given below.

- **INBOX**: Take the next input from the input stream.
- **OUTBOX**: Send the current element to the output stream.
- **ADD loc**: Add the current element with the one stored in memory specified by `loc`. The result is the new current element.
- **ADD [loc]**: Add the current element with the one referenced by the element in `loc`. The current element is then the result of this operation.
- **SUB loc** and **SUB [loc]**: Subtract the current element with the one specified by `loc` or referenced by it. The current element is then the result of this operation.
- **BUMP+ loc** and **BUMP+ [loc]**: Increment the value stored in `loc` or the one referenced by it. The current element is then the result of this operation.
- **BUMP- loc** and **BUMP- [loc]**: Decrement the value stored in `loc` or the one referenced by it. The current element is then the result of this operation.
- **JUMP id**: Jump the program to the specified `id`. The next instruction executed is the instruction after the jump.
- **JUMPZ id**: Jump the program to the specified `id` if the current element is 0. The next instruction executed is the instruction after the jump.
- **JUMPN id**: Jump the program to the specified `id` if the current element is negative. The next instruction executed is the instruction after the jump.

In order to specify the result of the jump instructions, we also have the following.

- `id ::` (the first `:` is part of the instruction). The jump instruction jumps to this point within the program.

The following program (see file `input-hrm.in1`) performs the above task.

```
-- HUMAN RESOURCE MACHINE PROGRAM --
```

```
a :
    INBOX
    COPYTO    0
    INBOX
    COPYTO    1
    COPYFROM  0
b :
    SUB      1
    JUMPN    c
    JUMPZ    d
    JUMP     b
c :
    ADD      1
d :
    OUTBOX
    JUMP     a
```

Note that the first line of the program always starts with the phrase “– HUMAN RESOURCE MACHINE PROGRAM –”. Also, instead of `num` for numeric constants and `id` for identifiers, we have the numbers 0, 1, etc. and identifiers a, b, c, etc. These are built into the grammar, but are limited. We therefore allow for single digit numeric constants and single digit identifiers. Also note that each token is explicitly differentiated between whitespaces. We will follow the same rules for expressing the input and production rules. The grammar for all Human Resource Machine programs is given in the file `grammar-hrm`. The format for the grammar is given in the next section. Please study this grammar to test your parser with other Human Resource Machine programs. Please also test your parser with the arithmetic expression grammar given in Section 1 along with several input strings. This grammar is given in file `grammar-expr`.

4 Input and Output

There are two inputs to the parser. The first one is a grammar and the second is the input string that needs to be parsed. The grammar is specified in the following format.

The first line of the grammar specifies two integers, the number of non-terminal symbols x and terminal symbols y . The next x lines contain non-terminal symbols, each of which is a string (can be a single letter or several letters). The next y lines contain terminal symbols, each of which is a string. Next, a single integer p indicates the number of production rules. Each rule is described using a pair of lines, the first of which indicates a non-terminal symbol, and the second line indicates its production string. Each token in the production string is guaranteed to be separated by whitespaces. See below for the grammar given in Section 1.

```
3 6
S
T
I
```

```

+
id
*
(
)
num
7
S
T
S
T + S
T
I
T
I * S
T
( S )
I
id
I
num

```

This is also in file `grammar-expr`. Please see the file `grammar-hrm` for the grammar of Human Resource Machine programs.

The input string we would like to parse can be expressed in a single line or in many lines. Each token in the input string is guaranteed to be separated by at least one whitespace. The Human Resource Machine program given in the previous section is a valid input string (see the file `input-hrm`).

The output of the program is a single line that says either “string is valid” or “string is invalid” depending on whether the input string belongs to the corresponding grammar or not. Please make sure that the output matches exactly and do not include other extraneous input or output. You may use the `diff` command to compare your output with the expected output shown in file `output-valid`.

5 Note on Data Structures

As a first step, you may consider parsing the input grammar file and storing a list of terminal symbols, non-terminal symbols, and the number of terminals, non-terminals and productions. You may store productions as a list of lists, or as a map. So `production["S"]` may be a list of lists, where each list is a production separated by its tokens. So `production["S"] = [["T"], ["T", "+", "S"]]`. This may allow you to compare each token of the input string with a symbol in the production string easily, especially if the input string is also stored as a list of tokens. Consequently, you can also store the productions as a list of list of lists, where `production[0]` provides a list of list of productions for the first non-terminal symbol "S", and `production[1]` provides production for the second non-terminal symbol and so on. Please feel free to experiment with different data structures for storing the input string and the productions.

Please see Section 6.3 for what library objects are allowed in your program.

6 Postamble

6.1 Execution

For this problem, we will use the command line to accept the grammar file, and get the input string from standard input. So, if you named your program `parser.py`, then we could run your program the following way.

```
python3 parser.py grammar-expr < input-expr
```

To grade your program, we use the `diff` utility to compare the output of your program against the expected output.

```
python3 parser.py grammar-expr < input-expr > out
diff out output-expr
```

Please make sure that you format your output correctly so that the output of `diff`

6.2 Submission

Please submit a single Python file named `parser.py` through Canvas.

6.3 Proper Use of Libraries

The following Python packages are allowed. In general, you are allowed to use libraries for input and output, math and random functions, but not packages that provide you substantially more code for the assignment. If you are in doubt, please consult your instructor.

Python tuples, lists, sets and dicts as defined in the Python language. Also allowed are `deque` and `heapq` from `collections`.

You may use these **Geeks for Geeks** resources for more information, or please check out the official language documentations.

Python: <https://www.geeksforgeeks.org/python-programming-language/>